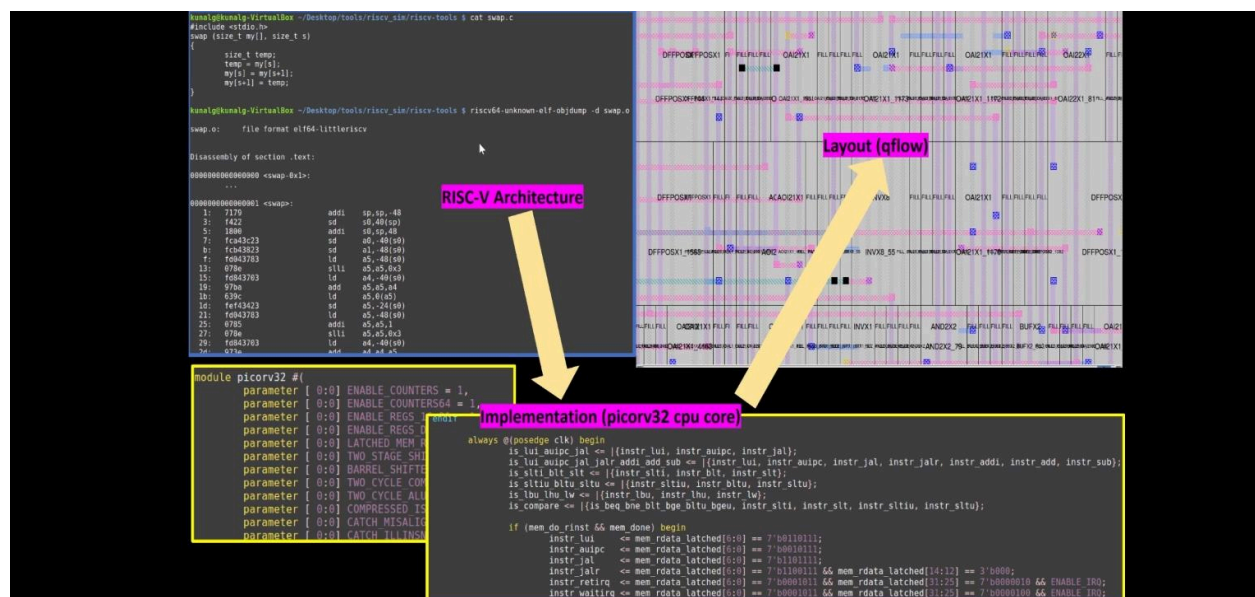


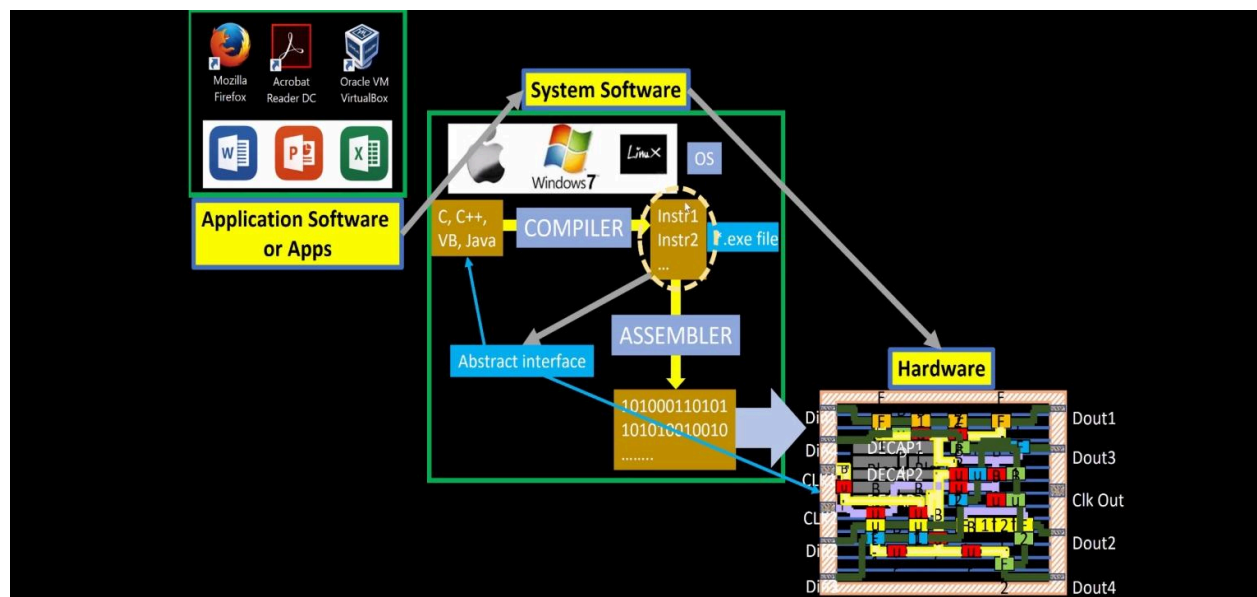
DAY 01

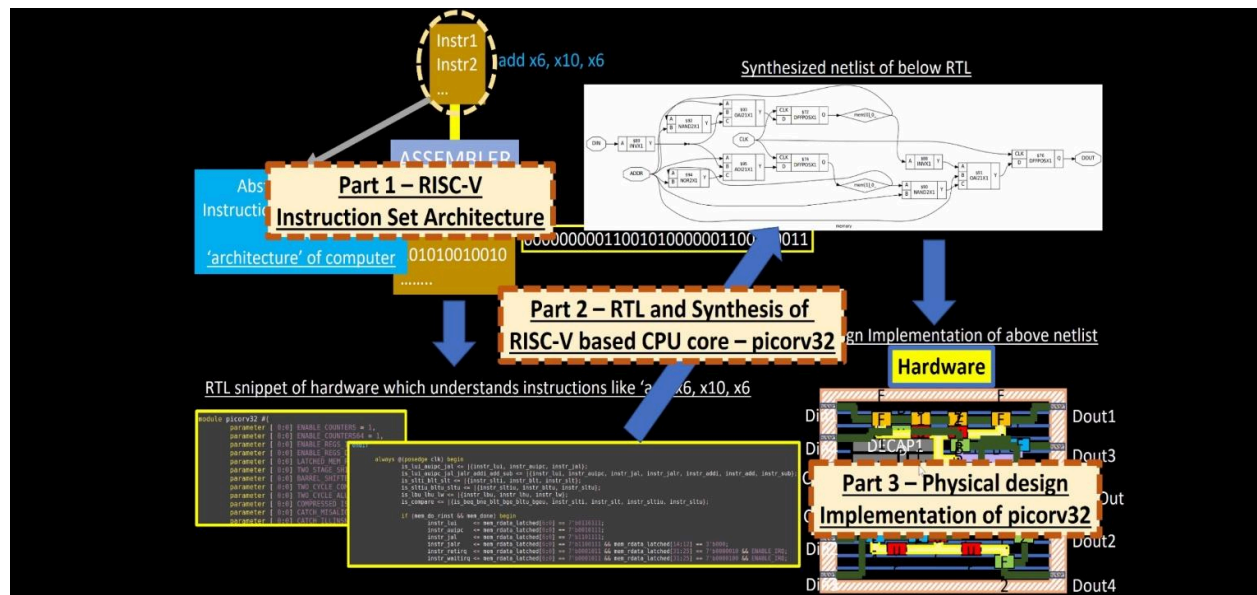
•INTRODUCTION TO RISC-V INSTRUCTION SET ARCHITECTURE:

RISC-V is an open-source instruction set architecture used to design modern processors. In this project, the complete flow of RISC-V processor development is studied from software level to hardware implementation. The process begins with generating RISC-V instructions from a program, followed by implementing the processor architecture using hardware design techniques. The designed processor is then converted into logic circuits and finally transformed into a physical chip layout for fabrication. This project helps in understanding the overall flow of processor design, including architecture, implementation, and physical realization of a RISC-V based CPU.



•A computer system mainly has three parts: application software, system software, and hardware. Application software includes programs we use daily such as Word, PowerPoint, Excel, Firefox, and VirtualBox. These applications help users do tasks like writing documents, browsing the internet, or creating presentations. System software acts like a bridge between the applications and the hardware. It includes operating systems such as Windows, Linux, and macOS, which manage the computer and allow applications to run properly. Programming languages like C, C++, Java, and VB are converted into machine-understandable instructions using tools such as a compiler and assembler. The compiler changes the program into lower-level instructions, and the assembler converts them into binary code (0s and 1s). Finally, the hardware executes these instructions. Hardware includes physical parts and electronic circuits inside the computer chip. In simple terms, application software helps the user, system software controls and translates commands, and hardware performs the actual work.





Part 1 – RISC-V Instruction Set Architecture (ISA)

This stage focuses on understanding the instruction set of the processor. The RISC-V ISA defines how the CPU communicates using instructions such as arithmetic, memory access, and logical operations. Assembly instructions are converted into binary machine code that the processor can understand and execute. This part mainly explains the basic architecture and working flow of the processor instructions.

Part 2 – RTL and Synthesis of RISC-V CPU Core

In this stage, the processor is designed using RTL (Register Transfer Level) coding, usually written in Verilog. The RTL describes how data moves inside the CPU and how different hardware blocks function together. After writing the RTL code, synthesis is performed to convert the design into a gate-level netlist made of logic gates and flip-flops.

Part 3 – Physical Design Implementation

This stage converts the synthesized netlist into an actual hardware layout. Different electronic components are placed and connected on the chip to create the final processor hardware. Physical design includes placement, routing, and optimization to ensure the chip works correctly and efficiently in real hardware form.

LAB 01

Steps to Run the Program in Ubuntu

1. Open Terminal

Press:

Ctrl + Alt + T

2. Go to Home Directory

```
cd
```

3. Create C File Using Leafpad

```
leafpad sum1ton.c
```

4. Type the Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i, sum = 0, n = 100;
```

```
    for(i = 1; i <= n; ++i)
```

```
    {
```

```
    sum += i;
}

printf("Sum of numbers from 1 to %d is %d", n, sum);

return 0;
}
```

5. Save the File

Click File → Save

Close Leafpad

6. Compile the Program

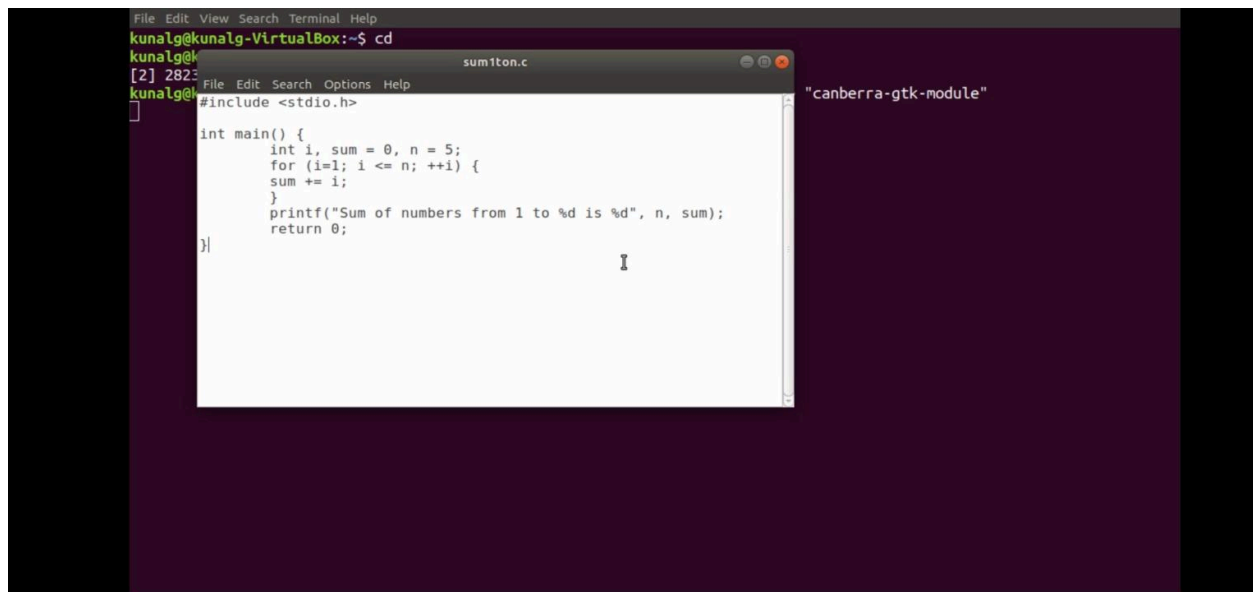
gcc sum1ton.c

7. Run the Program

./a.out

8. Output

Sum of numbers from 1 to 100 is 5050



```
File Edit View Search Terminal Help
kunalg@kunalg-VirtualBox:~$ cd
kunalg@kunalg:~$ gcc sum1ton.c
[2] 2822
kunalg@kunalg:~$ ./a.out
Sum of numbers from 1 to 100 is 5050
```

LAB CONTINUE

Commands with comments

Display the C program

```
cat sum1ton.c
```

Compile the program for RISC-V architecture

```
riscv64-unknown-elf-gcc -O1 -mabi=lp64 -march=rv64i -o sum1ton.o  
sum1ton.c
```

List the generated object file

```
ls -ltr sum1ton.o
```

Display assembly/disassembly code of object file

```
riscv64-unknown-elf-objdump -d sum1ton.o
```

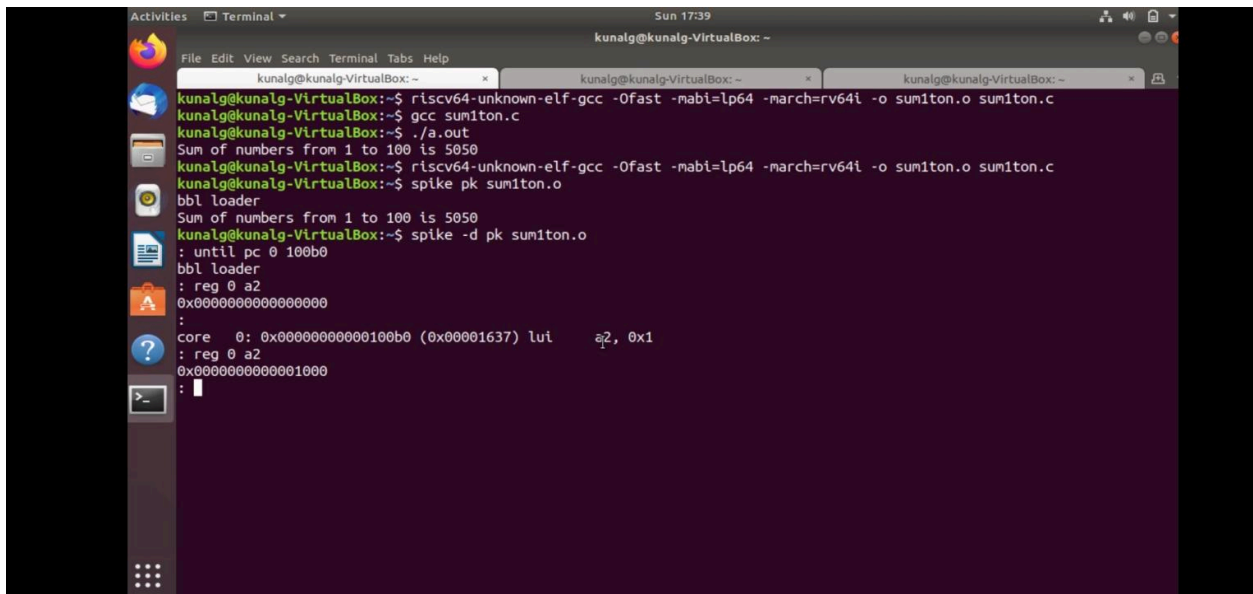
View assembly output page by page

```
riscv64-unknown-elf-objdump -d sum1ton.o | less
```

Compile with fast optimization

```
riscv64-unknown-elf-gcc -Ofast -mabi=lp64 -march=rv64i -o  
sum1ton.o sum1ton.c
```

Next continue:

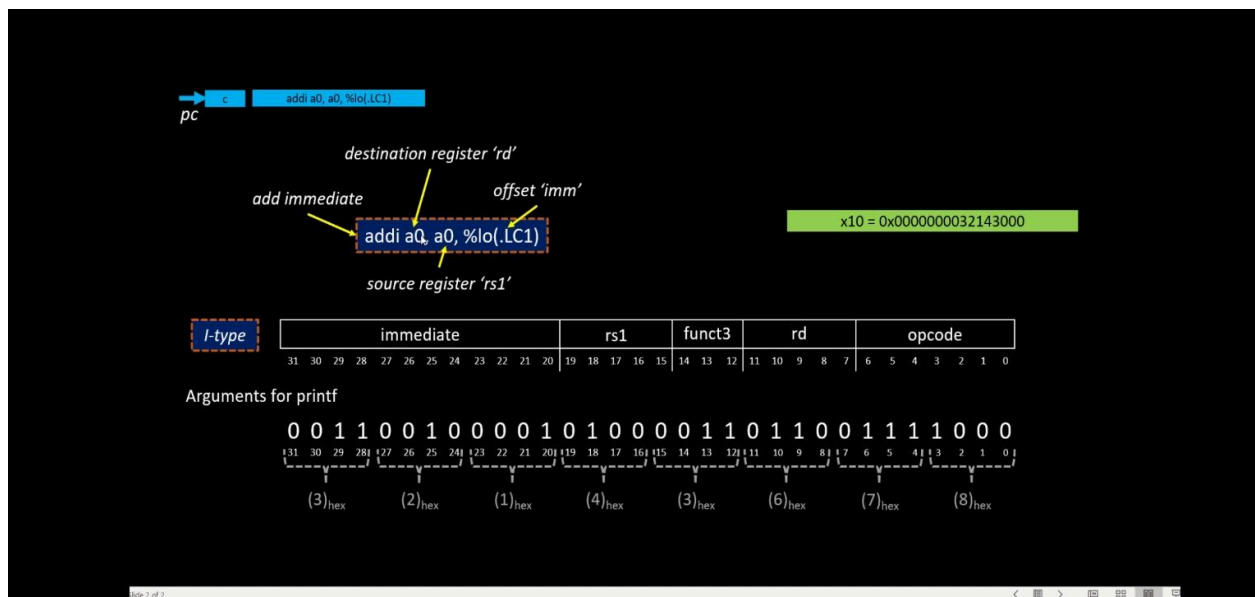
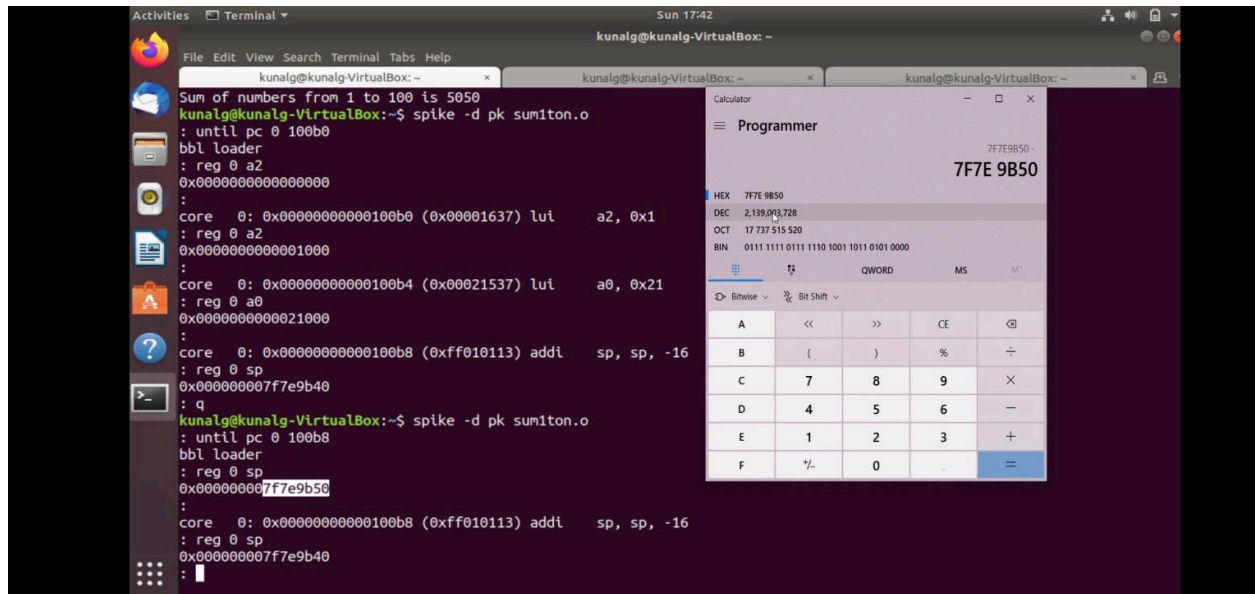


The screenshot shows a terminal window with the following commands and output:

```
kunalg@kunalg-VirtualBox:~$ riscv64-unknown-elf-gcc -Ofast -mabi=lp64 -march=rv64i -o sum1ton.o sum1ton.c
kunalg@kunalg-VirtualBox:~$ gcc sum1ton.c
kunalg@kunalg-VirtualBox:~$ ./a.out
Sum of numbers from 1 to 100 is 5050
kunalg@kunalg-VirtualBox:~$ riscv64-unknown-elf-gcc -Ofast -mabi=lp64 -march=rv64i -o sum1ton.o sum1ton.c
kunalg@kunalg-VirtualBox:~$ spike pk sum1ton.o
bbl loader
Sum of numbers from 1 to 100 is 5050
kunalg@kunalg-VirtualBox:~$ spike -d pk sum1ton.o
: until pc 0 100b0
bbl loader
: reg 0 a2
0x0000000000000000
:
core 0: 0x000000000000100b0 (0x00001637) lui    a2, 0x1
: reg 0 a2
0x0000000000001000
:
```

```
Activities Terminal Sun 17:36 kunalg@kunalg-VirtualBox: -
File Edit View Search Terminal Tabs Help
kunalg@kunalg-VirtualBox:~$ riscv64-unknown-elf-objdump -d sum1ton.o | less
kunalg@kunalg-VirtualBox:~$
```

```
Activities Terminal Sun 17:41 kunalg@kunalg-VirtualBox: -
File Edit View Search Terminal Tabs Help
kunalg@kunalg-VirtualBox:~$ spike pk sum1ton.o
bbl loader
Sum of numbers from 1 to 100 is 5050
kunalg@kunalg-VirtualBox:~$ spike -d pk sum1ton.o
: until pc 0 100b0
bbl loader
: reg 0 a2
0x0000000000000000
:
: core 0: 0x000000000000100b0 (0x00001637) lui a2, 0x1
: reg 0 a2
0x0000000000001000
:
: core 0: 0x000000000000100b4 (0x00021537) lui a0, 0x21
: reg 0 a0
0x00000000000021000
:
: core 0: 0x000000000000100b8 (0xff010113) addi sp, sp, -16
: reg 0 sp
0x000000007f7e9b40
: q
kunalg@kunalg-VirtualBox:~$ spike -d pk sum1ton.o
: until pc 0 100b8
bbl loader
: reg 0 s0
0x000000007f7e9b50
:
: core 0: 0x000000000000100b8 (0xff010113) addi sp, sp, -16
: reg
```



Let's start with basics

Floating-point addition

```

lui    a0,0x0
addi   sp,sp,-32
mv     a0,a0
sd     ra,24(sp)
sd     s0,16(sp)
auipc  ra,0x0
jalr   ra
lui    s0,0x0
addi   a1,sp,8
mv     a0,s0
auipc  ra,0x0
jalr   ra
lui    a0,0x0
mv     a0,a0
auipc  ra,0x0
jalr   ra
lui    a0,0x0
mv     a0,a0
auipc  ra,0x0
jalr   ra
addi   a1,sp,12
mv     a0,s0
auipc  ra,0x0
jalr   ra
lw     a5,12(sp)
lw     a1,8(sp)
lui    a0,0x0
mv     a0,a0
addw   a1,a1,a5
auipc  ra,0x0
jalr   ra
li     a0,10
auipc  ra,0x0
jalr   ra
ld     ra,24(sp)
ld     s0,16(sp)
li     a0,0
addi   sp,sp,32
ret
  
```

Pseudo Instructions

Base integer Instructions

RV64I

Multiply extension

RV64M

Single and double precision floating point extension

RV64F & RV64D

Application binary interface (ABI)

Memory allocation and stack pointer

Integer instructions

```

addi   sp,sp,-48
mv     a0,a0
sd     ra,40(sp)
sd     ra,32(sp)
sd     s1,24(sp)
lui    s0,0x0
auipc  ra,0x0
jalr   ra
addi   a1,sp,8
mv     a0,s0
auipc  ra,0x0
jalr   ra
lui    a0,0x0
mv     a0,a0
auipc  ra,0x0
jalr   ra
addi   a1,sp,12
mv     a0,s0
auipc  ra,0x0
jalr   ra
lw     a1,8(sp)
w     s0,12(sp)
lui    a0,0x0
vui    a0,a0
uivw   a1,s1,s0
uipc   ra,0x0
viwv   a1,s1,s0
vui    a0,0x0
vui    a0,a0
uipc   ra,0x0
vui    a0,0x0
uipc   ra,0x0
lui    a0,10
uipc   ra,0x0
ld     ra,40(sp)
ld     ra,32(sp)
ld     s1,24(sp)
li     a0,0
addi   sp,sp,48
ret
  
```

RV64F & RV64D

```

lui    a0,0x0
addi   sp,sp,-48
mv     a0,a0
sd     ra,40(sp)
sd     s0,32(sp)
sd     f5,16(sp)
lui    s0,0x0
auipc  ra,0x0
jalr   ra
addi   a1,sp,8
mv     a0,s0
auipc  ra,0x0
jalr   ra
lui    a0,0x0
mv     a0,a0
auipc  ra,0x0
jalr   ra
addi   a1,sp,12
mv     a0,s0
auipc  ra,0x0
jalr   ra
lw     f5,12(sp)
fw     f50,8(sp)
lui    a0,0x0
mvui   a0,a0
fmulw  f5,f50,f51
fdiv.s f50,f50,f51
fcvt.d.s f5,f5a5
fmv.x.d a1,f5a5
auipc  ra,0x0
jalr   ra
lui    a0,0x0
mv     a0,a0
fcvt.d.s f5a5,f50
fmv.x.d a1,f5a5
auipc  ra,0x0
jalr   ra
li     a0,10
auipc  ra,0x0
jalr   ra
ld     ra,40(sp)
ld     s0,32(sp)
ld     f50,24(sp)
li     a0,0
addi   sp,sp,48
ret
  
```

Pseudo instructions are shortcut commands used in RISC-V assembly language to make programming easier and simpler. These instructions are not directly executed by the processor. Instead, the assembler converts them into real machine instructions. They help programmers write readable and shorter code.

RV64I is the basic 64-bit instruction set of the RISC-V processor. It contains fundamental operations such as addition, subtraction, loading and storing data, branching, and logical operations. It forms the core foundation of the processor and allows the CPU to perform normal integer calculations and data handling.

Multiply Extension – RV64M

RV64M is an extension that adds multiplication, division, and remainder operations to the processor. Without this extension, the processor can only perform basic arithmetic operations. It improves the processor's ability to handle mathematical calculations efficiently.

Single and Double Precision Floating Point – RV64F & RV64D

RV64F and RV64D are floating-point extensions used for decimal number calculations. RV64F handles single-precision 32-bit floating-point numbers, while RV64D handles double-precision 64-bit floating-point numbers with higher accuracy. These extensions allow the processor to perform precise decimal arithmetic operations.

Application Binary Interface (ABI)

ABI is a set of rules that defines how programs and functions communicate with each other. It specifies how data is passed between functions, how return values are stored, and which registers are used during execution. ABI ensures compatibility between software, compilers, and the operating system.

Memory Allocation and Stack Pointer

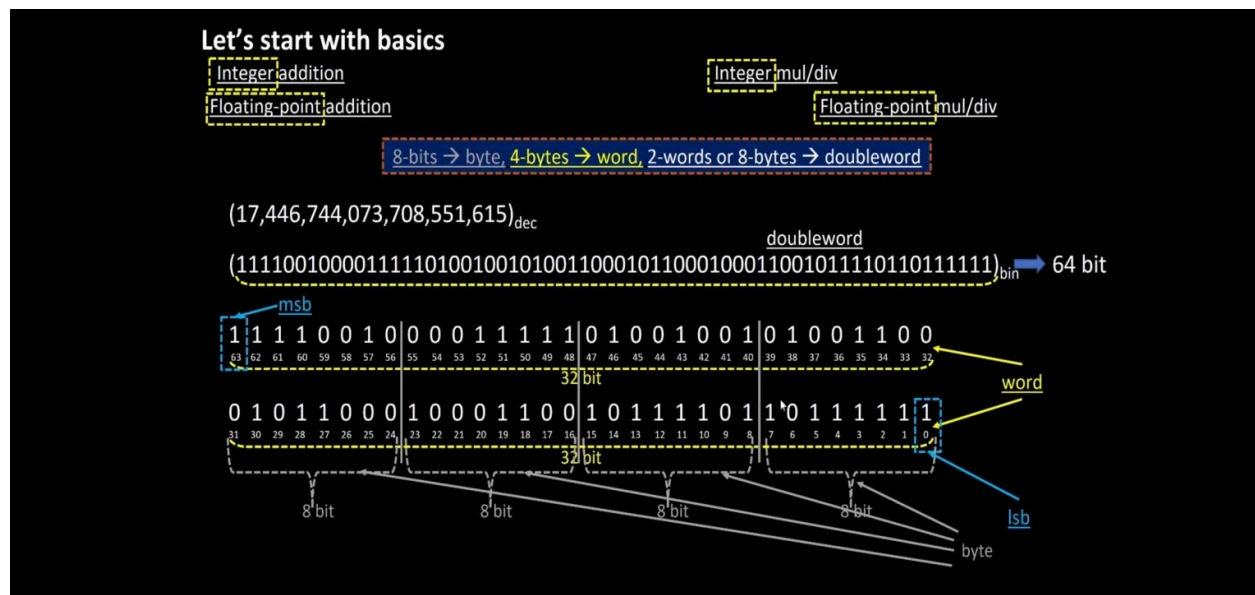
Memory allocation is the process of reserving memory space for variables and program data. The stack pointer (sp) is a special register that points to the top of stack memory. The stack stores temporary data, function information, and local variables during

program execution. The stack grows and shrinks as functions are called and completed.

Floating Point Addition

Floating-point addition is the process of adding decimal numbers in the processor. RISC-V uses floating-point registers and special instructions to perform these operations accurately. It allows the processor to handle decimal arithmetic efficiently and precisely.

Next continue



Let's start with basics

Integer addition

Floating-point addition

Integer mul/div

Floating-point mul/div

8-bits → byte, 4-bytes → word, 2-words or 8-bytes → doubleword

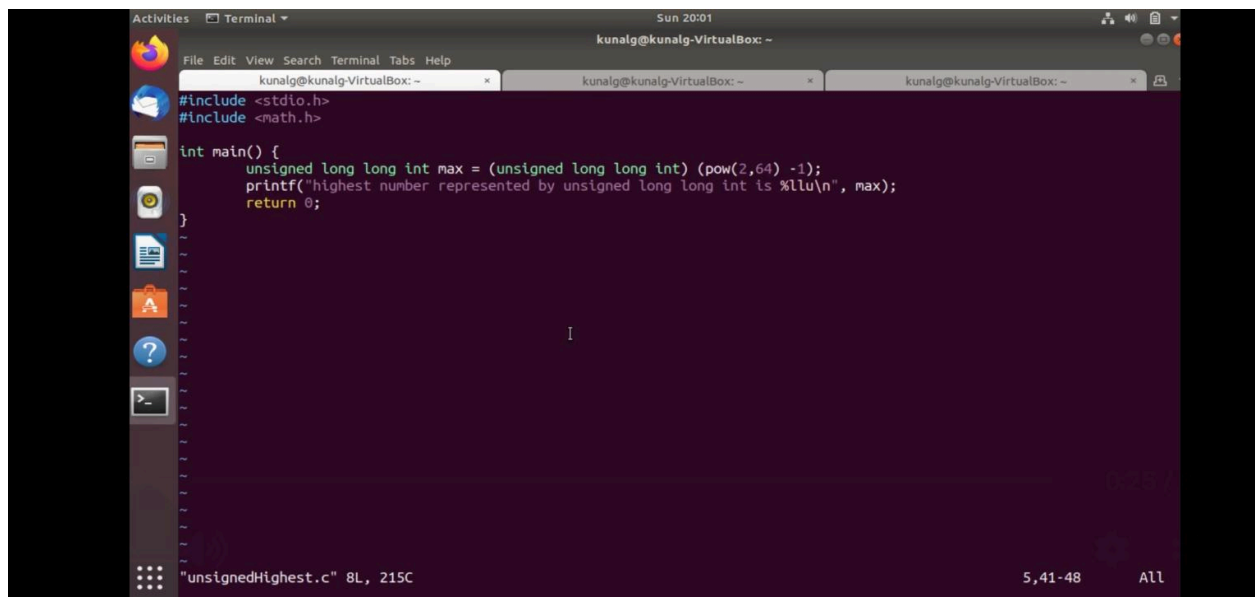
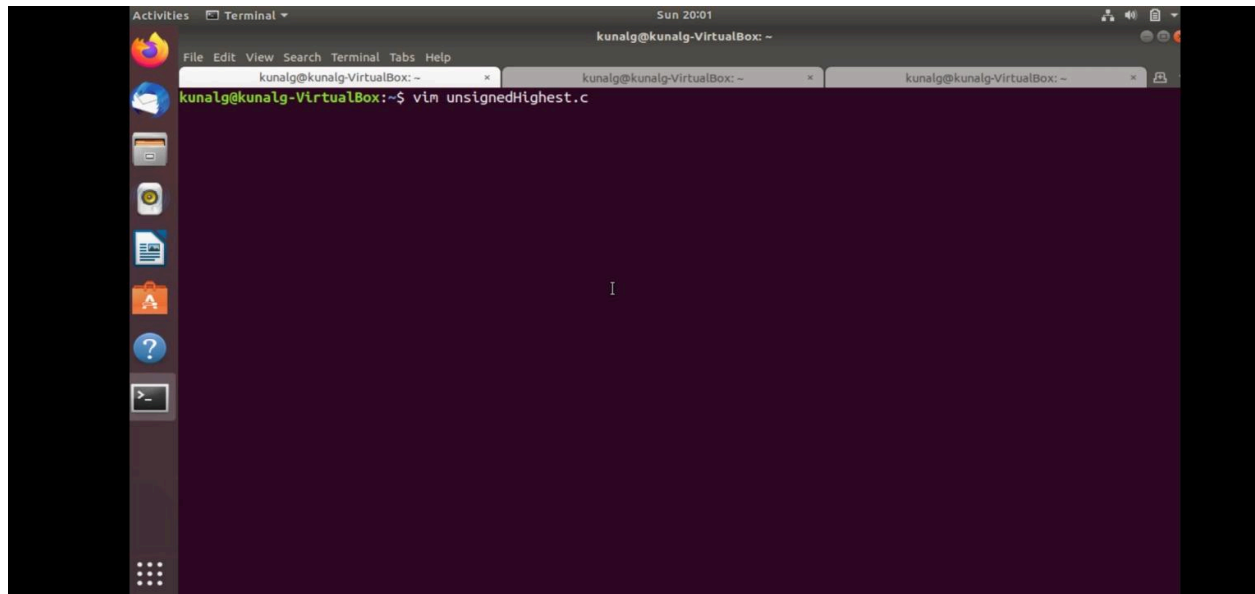
RISC-V doubleword can represent '0' to ' $2^{64} - 1$ ' unsigned numbers or positive numbers

$(17,446,744,073,708,551,615)_{\text{dec}}$

$(1111100100001111101001001010011000101100010001100101111011011111)_{\text{bin}} \rightarrow 64 \text{ bit}$

A computer stores and processes data using binary numbers made of only 0s and 1s. The smallest unit of data is called a bit. A group of 8 bits forms a byte. Larger groups are used to handle bigger numbers, such as a word which is 32 bits (4 bytes) and a doubleword which is 64 bits (8 bytes). In a 64-bit number, the leftmost bit is called the MSB (Most Significant Bit) because it has the highest value, while the rightmost bit is called the LSB (Least Significant Bit) because it has the lowest value. Processors like RISC-V use these binary values to perform operations such as addition, multiplication, and division for both integers and floating-point numbers. A 64-bit doubleword can represent very large unsigned values ranging from 0 to $2^{64} - 1$.

Example program:



```
Activities Terminal Sun 20:04 kunalg@kunalg-VirtualBox: ~
File Edit View Search Terminal Tabs Help
kunalg@kunalg-VirtualBox: ~ kunalg@kunalg-VirtualBox: ~ kunalg@kunalg-VirtualBox: ~
kunalg@kunalg-VirtualBox:~$ vim unsignedHighest.c
kunalg@kunalg-VirtualBox:~$ riscv64-unknown-elf-gcc -Ofast -mabi=lp64 -march=rv64i -o unsignedHighest
.c
kunalg@kunalg-VirtualBox:~$ spike pk unsignedHighest.o
bbl loader
highest number represented by unsigned long long int is 18446744073709551615
kunalg@kunalg-VirtualBox:~$ vim unsignedHighest.c
kunalg@kunalg-VirtualBox:~$ riscv64-unknown-elf-gcc -Ofast -mabi=lp64 -march=rv64i -o unsignedHighest
.c
```

Lab to find highest and lowest number

Data Type	Memory (bytes)	Format specifier
unsigned int	4	%u
int	4	%d
unsigned long long int	8	%llu
long long int	8	%lld